

Questions / Réponses — Soutenance MSPR E6.2+E6.4

ObRail Europe — Module IA Prédictif

BENBAHAZ Hakem Amine — EPSI — Juin 2026

DONNÉES

Q : D'où viennent vos données ? Les données proviennent de notre pipeline ETL ObRail : GTFS SNCF (France) et DB Germany Open Data, chargées dans PostgreSQL Azure via Azure Data Factory. Pour le module ML, j'ai utilisé le fichier `dessertes.csv` généré par l'ETL — 122 801 liaisons ferroviaires européennes.

Q : 122 801 lignes, c'est suffisant pour du ML ? Oui, largement. En données tabulaires structurées, scikit-learn donne de bons résultats à partir de 10 000 exemples. 122 801 lignes permet une cross-validation fiable et des résultats statistiquement solides.

Q : Quelles variables vous avez utilisées et pourquoi ? 6 variables finales : `distance_km`, `duree_h`, `frequence_hebdo`, `emissions_co2_gkm`, `type_ligne_enc`, `type_service_enc`. J'ai retiré `traction` car 99.7% des lignes sont électriques — aucune variance, aucune information utile. Garder une variable inutile ajoute du bruit sans améliorer les modèles.

Q : Comment vous avez géré les valeurs manquantes ? 59 valeurs manquantes dans `frequence_hebdo`. Je les ai remplacées par la médiane plutôt que de supprimer les lignes — on ne perd aucune donnée. La médiane est plus robuste que la moyenne face aux valeurs extrêmes.

RÉGRESSION

Q : Pourquoi prédire la durée du trajet ? C'est une information clé pour un observatoire ferroviaire : estimer la durée d'une liaison quand les horaires ne sont pas publiés, comparer l'efficacité des opérateurs, alimenter des outils de planification. C'est une valeur ajoutée métier concrète.

Q : $R^2=0.952$, ça veut dire quoi concrètement ? Le modèle explique 95.2% de la variabilité des durées. En pratique, l'erreur moyenne est de 6 minutes — pour estimer la durée d'un trajet sans connaître son horaire réel, c'est très acceptable.

Q : Pourquoi RandomForest est meilleur que LinearRegression ? La LinearRegression obtient $R^2=0.141$ — preuve que la relation n'est pas linéaire. Un TGV fait 450km en 2h, un régional fait 50km en 1h : la durée dépend du type de train, des arrêts, de la vitesse commerciale. RandomForest capture ces interactions non-linéaires naturellement grâce à ses arbres de décision.

Q : C'est quoi la cross-validation ? On découpe les données d'entraînement en 3 parties ($cv=3$). Le modèle est entraîné 3 fois, chaque fois sur 2 parties et évalué sur la 3e. Ça donne une estimation plus fiable des performances réelles et détecte le surapprentissage.

Q : C'est quoi le GridSearch ? GridSearchCV teste automatiquement toutes les combinaisons de paramètres. Ici : `max_depth` (10 ou 20) $\times$ `n_estimators` (50 ou 100) = 4 combinaisons. Résultat : `max_depth=20`, `n_estimators=100` sont les meilleurs.

Q : Différence entre MAE et RMSE ? MAE (Mean Absolute Error) = erreur moyenne absolue. RMSE (Root Mean Squared Error) = pareil mais pénalise davantage les grandes erreurs. Notre MAE=0.097h signifie 6 minutes d'erreur moyenne.

CLASSIFICATION

Q : C'est quoi la substituabilité avion→train ? Une variable binaire que j'ai créée : une liaison est "substituable" à un vol court-courrier si elle respecte 3 critères — distance ≥ 150 km, durée < 8 h, fréquence ≥ 2 /semaine. Sur 122 801 liaisons, 8 970 sont substituables (7.3%).

Q : Pourquoi 150km comme seuil et pas 300km ? Avec 300km, on n'obtenait que 1.1% de cas positifs. Notre dataset est dominé par des trains régionaux courts (médiane = 42km). Un déséquilibre de 98.9%/1.1% est trop extrême — les modèles prédisent tout le temps 0 et obtiennent 98.9% d'accuracy sans rien apprendre. À 150km on atteint 7.3%, gérable avec les bonnes techniques.

Q : Comment vous avez géré le déséquilibre des classes ? 3 mesures : split stratifié (maintenir le ratio 7.3%/92.7% dans train ET test), `class_weight='balanced'` qui pondère automatiquement la classe minoritaire, et métrique principale F1-score plutôt qu'accuracy.

Q : Pourquoi F1=1.0 ? C'est pas suspect ? Si, et je l'ai identifié. C'est du **data leakage** : la cible `substituable` a été construite à partir des mêmes features que l'entraînement (distance, durée, fréquence). Le modèle apprend la règle directement. En production réelle, la cible devrait venir de données externes indépendantes (trafic aérien Eurostat). C'est documenté comme limitation dans le rapport.

Q : C'est quoi le F1-score ? La moyenne harmonique de la précision et du rappel. Précision = parmi les liaisons prédites substituables, combien le sont vraiment. Rappel = parmi toutes les liaisons réellement substituables, combien on en détecte. Le F1 équilibre les deux — adapté aux classes déséquilibrées.

CLUSTERING

Q : C'est quoi le clustering et pourquoi non supervisé ? K-Means groupe les données sans étiquettes prédéfinies — on ne dit pas au modèle quelles catégories chercher, il découvre la structure lui-même. Non supervisé = pas de "bonne réponse" pendant l'entraînement. Utile pour explorer des données dont on ne connaît pas encore les groupes naturels.

Q : Comment vous avez choisi k=5 ? Automatiquement par silhouette score. On teste k=2, 3, 4, 5 et on mesure la cohésion des clusters. Le silhouette score de k=5 (0.604) est le plus élevé donc on garde 5 clusters.

Q : C'est quoi le silhouette score ? Une mesure entre -1 et +1. Pour chaque point, il mesure combien il est proche des autres points de son cluster et éloigné des autres clusters. Proche de 1 = excellent. Notre 0.604 indique des groupes bien définis.

Q : Pourquoi normaliser les données avant K-Means ? K-Means calcule des distances euclidiennes. Sans normalisation, `distance_km` (0 à 30 000) écraserait `duree_h` (0 à 22). Le `StandardScaler` ramène tout à moyenne=0, écart-type=1 pour que chaque variable contribue équitablement.

Q : Le cluster avec 17 720km de distance moyenne, c'est quoi ? Des anomalies dans les données — aucun train européen ne parcourt 17 720km. Ce sont des erreurs dans les données GTFS sources. Le clustering les a automatiquement isolées sans qu'on les ait ciblées — le ML a détecté des erreurs de données qu'une analyse manuelle aurait manquées.

API ET DÉPLOIEMENT

Q : Pourquoi intégrer le ML dans une API REST ? Un fichier .pkl Python n'est accessible que depuis Python. Via une API REST, n'importe quel client (dashboard JavaScript, mobile, Excel) peut interroger les modèles avec une requête HTTP standard. C'est ce qui rend le ML réellement utilisable en production.

Q : Comment les modèles sont chargés en production ? Lazy loading avec cache mémoire : les .pkl sont chargés au premier appel et gardés en mémoire. Les appels suivants sont instantanés. Si les fichiers sont absents, l'API retourne un 503 Service Unavailable avec un message explicite.

Q : Comment vous sécurisez le /predict ? Par le middleware API Key existant — tous les endpoints `/api/v1/*` nécessitent le header `X-API-Key`. Sans clé valide : 401 Unauthorized.

TESTS

Q : Pourquoi 20 tests ? Qu'est-ce qu'ils vérifient ? 4 classes : chargement des modèles (4), régression (6), classification (5), clustering (5). Les tests ne vérifient pas juste "ça ne plante pas" — ils vérifient la logique métier : un train de 30km ne doit pas être substituable, un TGV Paris-Lyon doit l'être, plus la distance est grande plus la durée doit être longue.

BENCHMARK CLOUD

Q : Quelle solution cloud vous recommandez et pourquoi ? Azure ML. Notre infrastructure utilise déjà Azure Data Factory et Azure PostgreSQL. Rester dans le même écosystème simplifie les accès, la facturation et la conformité RGPD. Azure ML intègre aussi MLflow nativement. Coût estimé : 2-5€/mois pour notre volume.

Q : C'est quoi MLflow ? Un outil pour tracker les expériences ML : chaque entraînement enregistre automatiquement les paramètres, les métriques et les modèles. Ça permet de comparer des dizaines d'expériences et de reproduire les meilleurs résultats.

QUESTIONS GÉNÉRALES

Q : Si vous refaisiez ce projet, vous changeriez quoi ? 3 choses : définir substituable à partir de données externes (trafic aérien Eurostat) pour éviter le data leakage, filtrer les distances aberrantes avant le clustering, et ajouter des valeurs SHAP pour expliquer chaque prédiction individuellement.

Q : Le modèle peut se tromper ? Oui — le RandomForest régression a une MAE de 6 minutes en moyenne, il peut se tromper davantage sur des cas rares. En production on ajouterait un monitoring de dérive pour détecter quand les données changent et déclencher un réentraînement automatique.

Q : C'est quoi le surapprentissage et comment vous l'avez évité ? Le surapprentissage c'est quand le modèle apprend par cœur les données d'entraînement mais généralise mal sur de nouvelles données. On l'évite avec la cross-validation, un test set séparé (20%) jamais utilisé pendant l'entraînement, et des hyperparamètres comme `max_depth` qui limitent la complexité.

Q : Quelle est la différence entre les 3 familles de modèles ? Régression : prédit une valeur continue (durée en heures). Classification : prédit une catégorie (substituable ou non). Clustering : découvre des groupes sans étiquettes prédéfinies. Régression et classification sont supervisées (on leur donne les bonnes réponses), clustering est non supervisé.

Q : Comment vous choisissez entre RandomForest et XGBoost ? On compare leurs performances en cross-validation sur le même dataset. Ici RandomForest gagne ($R^2=0.952$ vs 0.913) — il est plus robuste aux outliers présents dans nos données. En pratique le résultat dépend toujours du dataset, c'est pour ça qu'on teste les deux systématiquement.